A Lab Rreport on
Raspberry Pi 5 Hailo AI Detection via MQTT

Group Members: Milan Khadka ,Udit Bhattarai and Dipen Gaihre

Note: I did this lab with this group because my team member was not present.

Course: Controllers and Electronics

Laboratory Work: Lab 7 – Raspberry Pi 5 Hailo AI Detection via MQTT

Platform: Raspberry Pi Zero 2W & Raspberry Pi 5

Libraries: pigpio, paho-mqtt, time, Json

Date: 15.12.2025

Note: This experiment was carried out by a group of two students. AI tools were used as writing assistants to improve the clarity of the report and to support code generation during the lab while performing the different tasks.

1. Objectives:

The objective of this lab was to understand how AI-generated data can be transmitted using MQTT and processed on an IoT edge device.

In this experiment, a Raspberry Pi 5 running a Hailo AI person-detection model publishes detection results via MQTT. A Raspberry Pi Zero 2 W subscribes to these messages, parses the received JSON data, and reacts to the detection results by controlling LEDs connected to GPIO pins.

The expected outcome was a working system where person detection events are visualized in real time using LEDs, based on detection confidence and connection status.

2. Theory and Background:

This section explains the theoretical background, as required in the “A good lab report – Hyvä laboratorioseloste” guidelines.

2.1 MQTT Communication:

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe protocol designed for IoT applications. Devices do not communicate directly with each other; instead, they exchange messages via a broker using topics. This makes MQTT efficient, scalable, and suitable for low-power and distributed systems.

In this lab:

- The **Raspberry Pi 5 + Hailo Pipeline** acts as a **publisher**
- The **Raspberry Pi Zero** 2 W acts as a **subscriber**
- The **broker** IP address is **10.10.200.15**
- Topics used are:
  - **melli/sensor/value**
  - **melli/sensor/status**

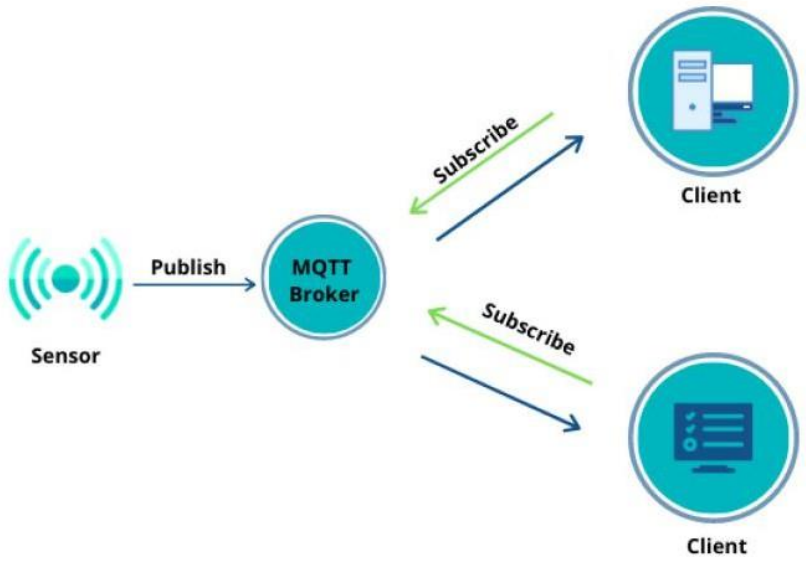
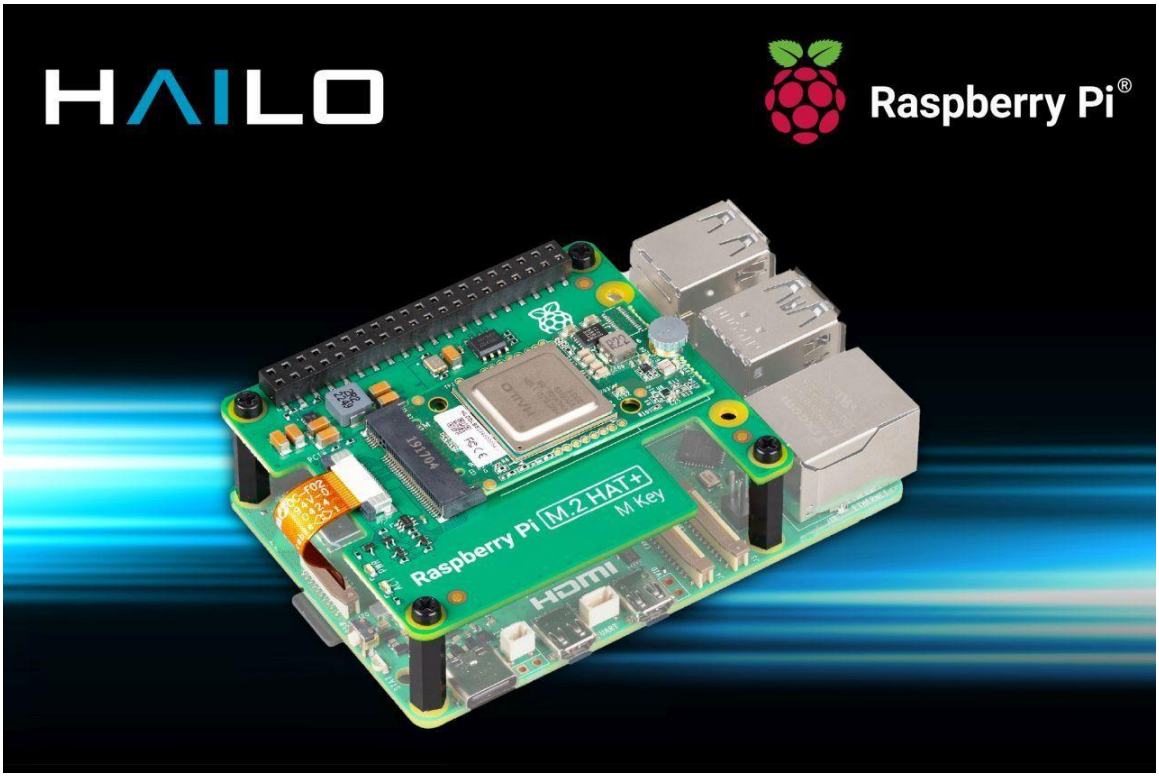


Figure: MQTT Subscribe and Publish method with Broker

2.2 Raspberry Pi 5 with Hailo AI Kit:

The Raspberry Pi 5 can be equipped with an optional **Hailo-8L AI acceleration module**, which provides dedicated hardware for neural-network processing. Instead of running AI models only on the Raspberry Pi CPU or GPU, the Hailo accelerator is designed specifically for deep-learning inference. This allows AI workloads to run much faster while consuming significantly less power.

Because of these characteristics, the Hailo AI accelerator is well suited for edge-computing applications, such as smart cameras, IoT automation systems, robotics, and real-time object detection. Running AI at the edge also reduces latency and avoids sending large amounts of data to the cloud.



The Hailo module is installed on the Raspberry Pi 5 using the **PCIe FPC connector** available on the board. Once installed and configured, the Hailo runtime environment allows the execution of specially compiled AI models in the form of **HEF (Hailo Executable File)** files.

The main advantages of using the Hailo AI accelerator with Raspberry Pi 5 are:

- Real-time performance for AI workloads

- Low power consumption compared to CPU-based inference
- High detection accuracy using optimized models
- Simple deployment using prebuilt pipelines and example scripts
- Compatibility with standard Raspberry Pi camera modules

In this lab, the Raspberry Pi 5 equipped with the Hailo AI Kit runs an AI inference pipeline that detects people in a camera image. The detection results are published via MQTT. The AI pipeline itself is not modified by the students. Instead, Raspberry Pi Zero 2 W devices are used to subscribe to the published MQTT messages and visualize the detection results using LEDs.

2.3 Person Detection with Hailo AI (Example Model and Pipeline):

Hailo provides ready-made example pipelines for the Raspberry Pi 5, including object-detection demos.

These example pipelines demonstrate how computer vision inference can be performed efficiently on the edge using the Hailo accelerator.

The Hailo Raspberry Pi 5 examples output detection results that include **object labels**, **confidence scores**, and other metadata such as bounding boxes. In this lab, the detection data is transmitted in a simplified JSON format using MQTT. An example of such a message is shown below:

```
{"label": "person", "confidence": 0.82}
```

The **label** field describes the detected object, while the **confidence** value represents how certain the AI model is about the detection. The confidence value ranges from 0 to 1, where values closer to 1 indicate higher certainty.

The task for us in this lab is to:

1. Receive the MQTT messages published by the Raspberry Pi 5
2. Interpret the label and confidence values from the JSON data
3. Trigger LED indicators on the Raspberry Pi Zero 2 W according to the detection results

This approach demonstrates how AI inference, IoT communication, and physical outputs can be combined into a complete edge-AI system.

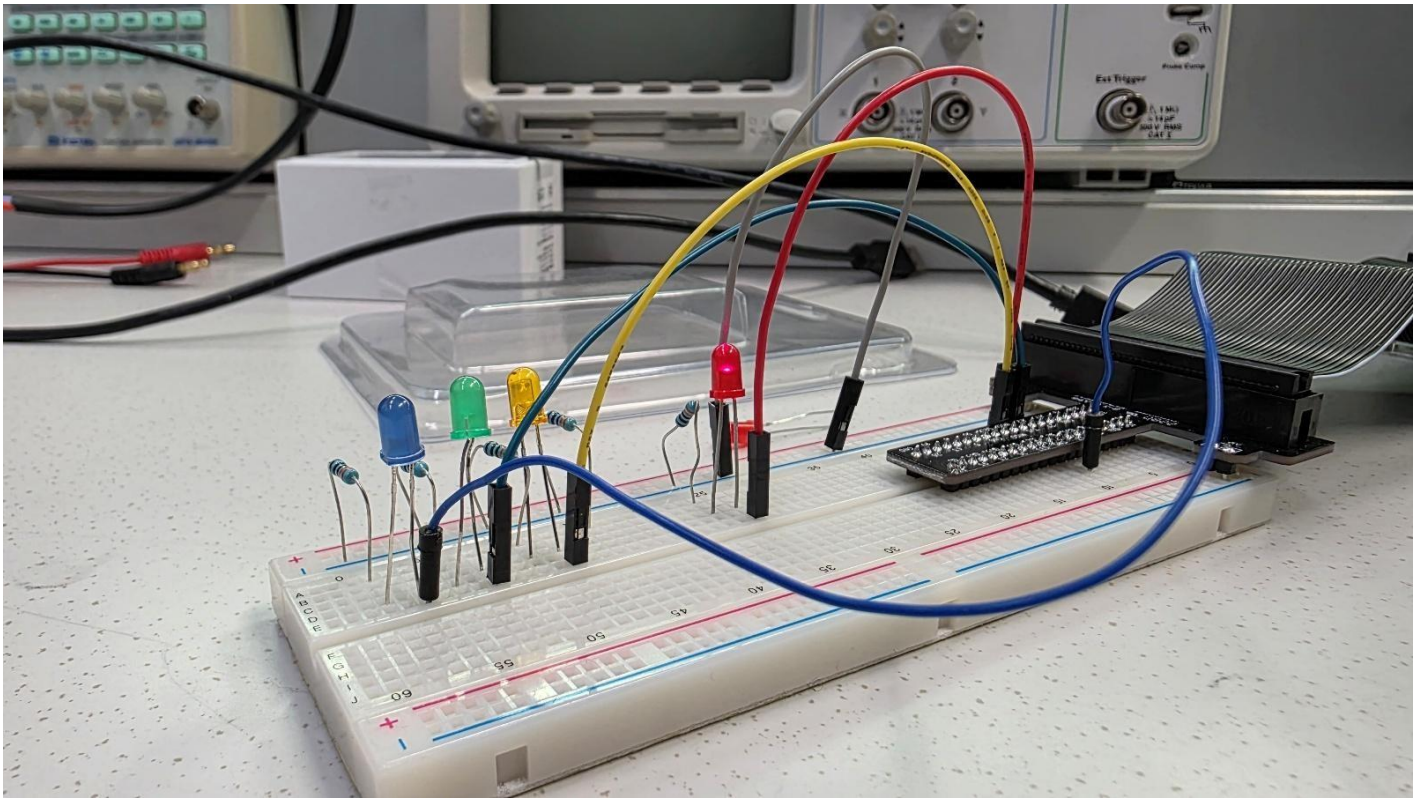
3. Equipment, Materials and Methods:

Component	Quantity
Raspberry Pi Zero 2 W	1
Raspberry Pi 5 with Hailo AI Kit	1
Breadboard	1
LEDs (Red, Green, Yellow, Blue)	4
Resistors (220–330 Ω)	4
Jumper wires	As needed
MQTT Broker	1
Python libraries	paho-mqtt, json, time, pigpio

3.2 Wiring Setup:

The Teacher already has the Raspberry Pi 5 with Hailo AI Kit setup. So, for us we only have setup for Raspberry Pi Zero 2W with LEDs and Resistors.

The photo of the setup is below:



4. Task Perform in Lab 7:

In this lab 7 we successfully performed the 4 tasks and one Research task. The screenshot of the code we used in different tasks are listed below with their explanation and output.

<u>Task 1:</u>

Understanding the MQTT Data

(1) In this lab, the Raspberry Pi 5 running the Hailo AI pipeline publishes detection results using MQTT messages. These messages are sent in JSON format, which makes the data structured and easy to process in Python.

(2) Each received MQTT message contains the following fields:

- **label** – describes the detected object (for example, "person")
- **confidence** – a numerical value that indicates how certain the AI model is about the detection

An example of a received message is shown below:

```
{"label": "person", "confidence": 0.82}
```

(3) The confidence value represents the probability that the detected object truly belongs to the given label. Its value ranges from 0.0 to 1.0, where values closer to 1.0 indicate higher certainty and more reliable detections. Lower values indicate less confidence and a higher possibility of incorrect detection.

Based on this data, LED indicators can be triggered according to predefined rules. In this lab, the following logic was used:

- If no person is detected, a green LED is turned ON
- If a person is detected with high confidence (≥ 0.75), a red LED is turned ON
- If a person is detected with low confidence (< 0.75), a yellow LED is turned ON
- If the MQTT connection is lost, a blue LED indicates the connection problem

This task helped in understanding how AI-generated data is structured and how it can be translated into meaningful actions in an IoT system.

Task 2: Subscribing to the MQTT Topic

In this task, a subscriber program was created on the **Raspberry Pi Zero 2 W** using the paho-mqtt Python library. The purpose of this script was to connect to the MQTT broker, subscribe to the specified topics, and display all received messages in the terminal.

Code used for Task 2:

```
labuser@RiiA103RPiZe21: ~/p
  GNU nano 8.4                                                    lab7_task2.py
import paho.mqtt.client as mqtt

BROKER = "10.10.200.15"
PORT = 1883
TOPICS = [("melli/sensor/value", 0), ("melli/sensor/status", 0)]

def on_connect(client, userdata, flags, reason_code, properties=None):
    print("Connected with result code:", reason_code)
    # Subscribe when connected
    for t, q in TOPICS:
        client.subscribe(t)
        print(f"Subscribed to topic: {t}")

def on_message(client, userdata, msg):
    print("---- MQTT MESSAGE ----")
    print("Topic:   ", msg.topic)
    print("Payload: ", msg.payload.decode())
    print("-----------------------")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, keepalive=60)

print("Starting MQTT loop. Press Ctrl+C to stop.")
client.loop_forever()
```

Output in Terminal Task 2:

```
labuser@RiiA103RPiZe21:~/pigpio/milan_udit $ python3 lab7_task2.py
Starting MQTT loop. Press Ctrl+C to stop.
Connected with result code: Success
Subscribed to topic: melli/sensor/value
Subscribed to topic: melli/sensor/status
---- MQTT MESSAGE ----
Topic:    melli/sensor/status
Payload:  {"status": "online"}
----------------------
---- MQTT MESSAGE ----
Topic:    melli/sensor/value
Payload:  {"label": "person", "confidence": 0.8393012881278992}
----------------------
---- MQTT MESSAGE ----
Topic:    melli/sensor/value
Payload:  {"label": "person", "confidence": 0.9633731245994568}
----------------------
---- MQTT MESSAGE ----
Topic:    melli/sensor/value
Payload:  {"label": "person", "confidence": 0.9214873313903809}
----------------------
---- MQTT MESSAGE ----
Topic:    melli/sensor/value
Payload:  {"label": "person", "confidence": 0.9443341493606567}
----------------------
---- MQTT MESSAGE ----
Topic:    melli/sensor/value
Payload:  {"label": "person", "confidence": 0.9557575583457947}
----------------------
---- MQTT MESSAGE ----
Topic:    melli/sensor/value
Payload:  {"label": "person", "confidence": 0.9481419324874878}
----------------------
---- MQTT MESSAGE ----
```

<u>Task 3</u>:

Parse JSON Data into a Dictionary

In this task incoming MQTT payloads were decoded and converted into Python dictionaries using `json.loads()`. The script printed:

- The raw MQTT message
- The parsed dictionary
- The label and confidence values separately

This made the data easier to use for decision-making logic.

Code used for Task 3:

```
  GNU nano 8.4                                                    lab7_task3.py
import paho.mqtt.client as mqtt
import json

BROKER = "10.10.200.15"
PORT = 1883
TOPICS = [("melli/sensor/value", 0), ("melli/sensor/status", 0)]

def on_connect(client, userdata, flags, reason_code, properties=None):
    print("Connected with result code:", reason_code)
    for t, q in TOPICS:
        client.subscribe(t)
        print(f"Subscribed to topic: {t}")

def on_message(client, userdata, msg):
    raw_payload = msg.payload.decode()
    print("\n==== NEW MESSAGE ====")
    print("Raw message:", raw_payload)

    try:
        data = json.loads(raw_payload)
        print("As dictionary:", data)

        # Safely access label & confidence
        label = data.get("label")
        confidence = data.get("confidence")

        print("Label:", label)
        print("Confidence:", confidence)

    except json.JSONDecodeError:
        print("Could not decode JSON! Payload might not be JSON.")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, keepalive=60)
print("Starting MQTT loop. Press Ctrl+C to stop.")
client.loop_forever()
```

Output in Terminal Task 3:

```
labuser@RiiA103RPiZe21:~/pigpio/milan_udit $ python3 lab7_task3.py
Starting MQTT loop. Press Ctrl+C to stop.
Connected with result code: Success
Subscribed to topic: melli/sensor/value
Subscribed to topic: melli/sensor/status

==== NEW MESSAGE ====
Raw message: {"status": "online"}
As dictionary: {'status': 'online'}
Label: None
Confidence: None

==== NEW MESSAGE ====
Raw message: {"label": "person", "confidence": 0.8910249471664429}
As dictionary: {'label': 'person', 'confidence': 0.8910249471664429}
Label: person
Confidence: 0.8910249471664429

==== NEW MESSAGE ====
Raw message: {"label": "person", "confidence": 0.8320346474647522}
As dictionary: {'label': 'person', 'confidence': 0.8320346474647522}
Label: person
Confidence: 0.8320346474647522

==== NEW MESSAGE ====
Raw message: {"label": "person", "confidence": 0.8834093809127808}
As dictionary: {'label': 'person', 'confidence': 0.8834093809127808}
Label: person
Confidence: 0.8834093809127808

==== NEW MESSAGE ====
Raw message: {"label": "person", "confidence": 0.9010680913925171}
As dictionary: {'label': 'person', 'confidence': 0.9010680913925171}
Label: person
Confidence: 0.9010680913925171
```

Task 4:

LED Indicator Logic

Four LEDs were connected to GPIO pins with current-limiting resistors. The following logic was implemented:

- Green LED: No person detected
- Red LED: Person detected with high confidence (≥ 0.75)
- Yellow LED: Person detected with low confidence (< 0.75)
- Blue LED: MQTT connection lost (optional)

The Python script was extended to control GPIO outputs based on received detection data.

Code used for Task 4: codes are little bit longer in this task, so I separate them into two for easy reading. We also change some values in code so that we can play with LEDs and camera with respect to values.

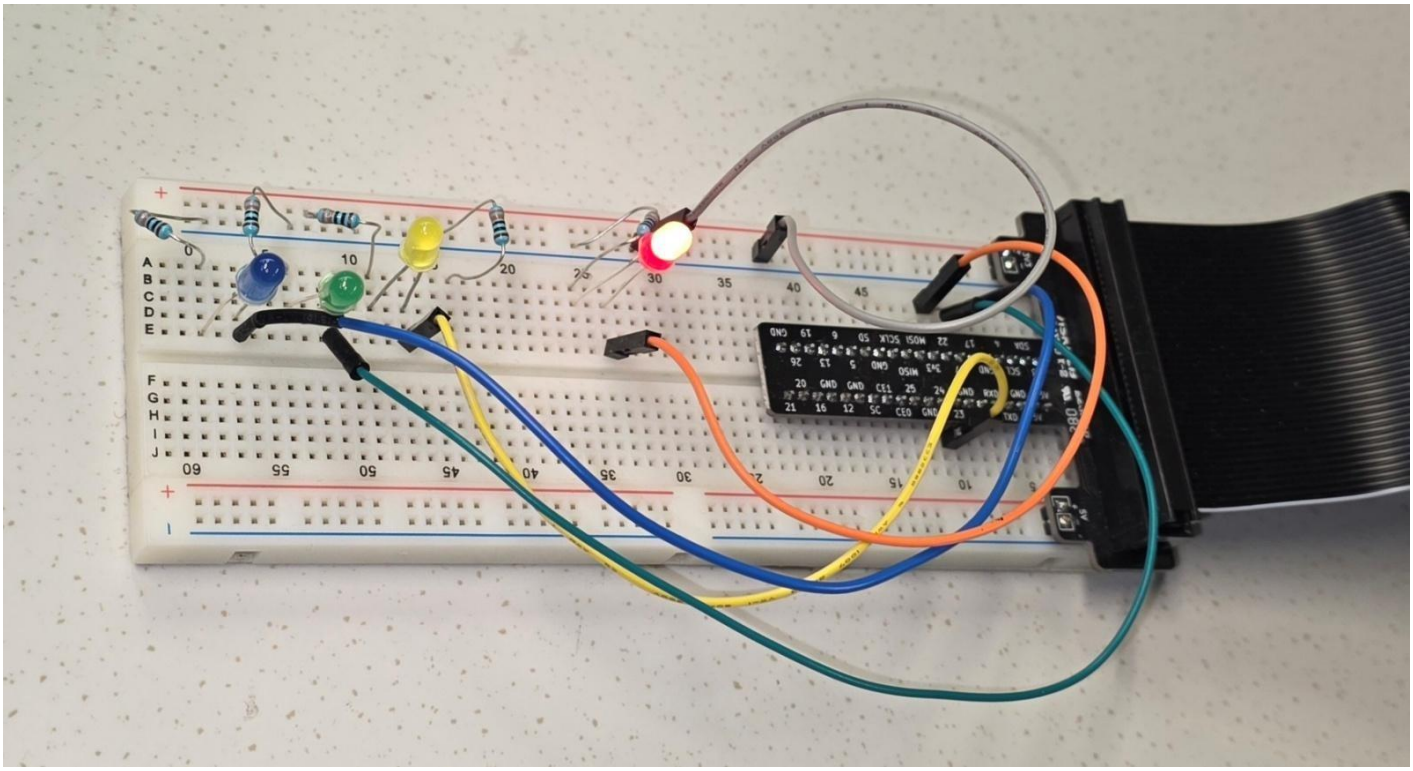
```
labuser@RiiA103RPiZe21: ~/p
  GNU nano 8.4                                                    lab7_task4_1.py
import paho.mqtt.client as mqtt
import json
import RPi.GPIO as GPIO
import time

# -------- GPIO SETUP --------
GPIO.setmode(GPIO.BCM)

GREEN = 17   # no person
RED = 27     # high confidence
YELLOW = 22  # low confidence
BLUE = 23    # connection lost (optional)

LEDS = [GREEN, RED, YELLOW, BLUE]

for pin in LEDS:
    GPIO.setup(pin, GPIO.OUT)
    GPIO.output(pin, GPIO.LOW)

def all_off():
    for pin in LEDS:
        GPIO.output(pin, GPIO.LOW)

# -------- MQTT SETTINGS --------
BROKER = "10.10.200.15"
PORT = 1883
TOPIC = "melli/sensor/value"

# -------- MQTT CALLBACKS --------
def on_connect(client, userdata, flags, reason_code, properties=None):
    print("Connected to MQTT")
    client.subscribe(TOPIC)
    GPIO.output(BLUE, GPIO.LOW)   # connection OK

def on_disconnect(client, userdata, reason_code, properties=None):
    print("MQTT disconnected!")
    all_off()
    GPIO.output(BLUE, GPIO.HIGH)  # blue LED ON for disconnect

def handle_detection(label, confidence):
    all_off()

    if confidence <= 0.40:
        GPIO.output(GREEN, GPIO.HIGH)
        print("GREEN → No person detected")
        return

^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location  M-U Undo      M-A Set Mark   M-] To Bracket  M-B
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy       ^B Where Was    M-F
```

```
labuser@RiiA103RPiZe21: ~/p
GNU nano 8.4                                                    lab7_task4_1.py

    if confidence <= 0.40:
        GPIO.output(GREEN, GPIO.HIGH)
        print("GREEN → No person detected")
        return

    if confidence >= 0.90:
        GPIO.output(RED, GPIO.HIGH)
        print("RED → High confidence person detected")
    else:
        GPIO.output(YELLOW, GPIO.HIGH)
        print("YELLOW → Low confidence person detected")

def on_message(client, userdata, msg):
    print("\nMQTT Message:", msg.payload.decode())

    try:
        data = json.loads(msg.payload.decode())
        label = data.get("label")
        confidence = data.get("confidence")

        print("Label:", label)
        print("Confidence:", confidence)

        handle_detection(label, confidence)

    except Exception as e:
        print("Error parsing message:", e)

# -------- MAIN --------
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_disconnect = on_disconnect
client.on_message = on_message

try:
    client.connect(BROKER, PORT)
    print("Listening for detections...")
    client.loop_forever()

except KeyboardInterrupt:
    print("Exiting...")

finally:
    all_off()
    GPIO.cleanup()

^G Help      ^O Write Out   ^F Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

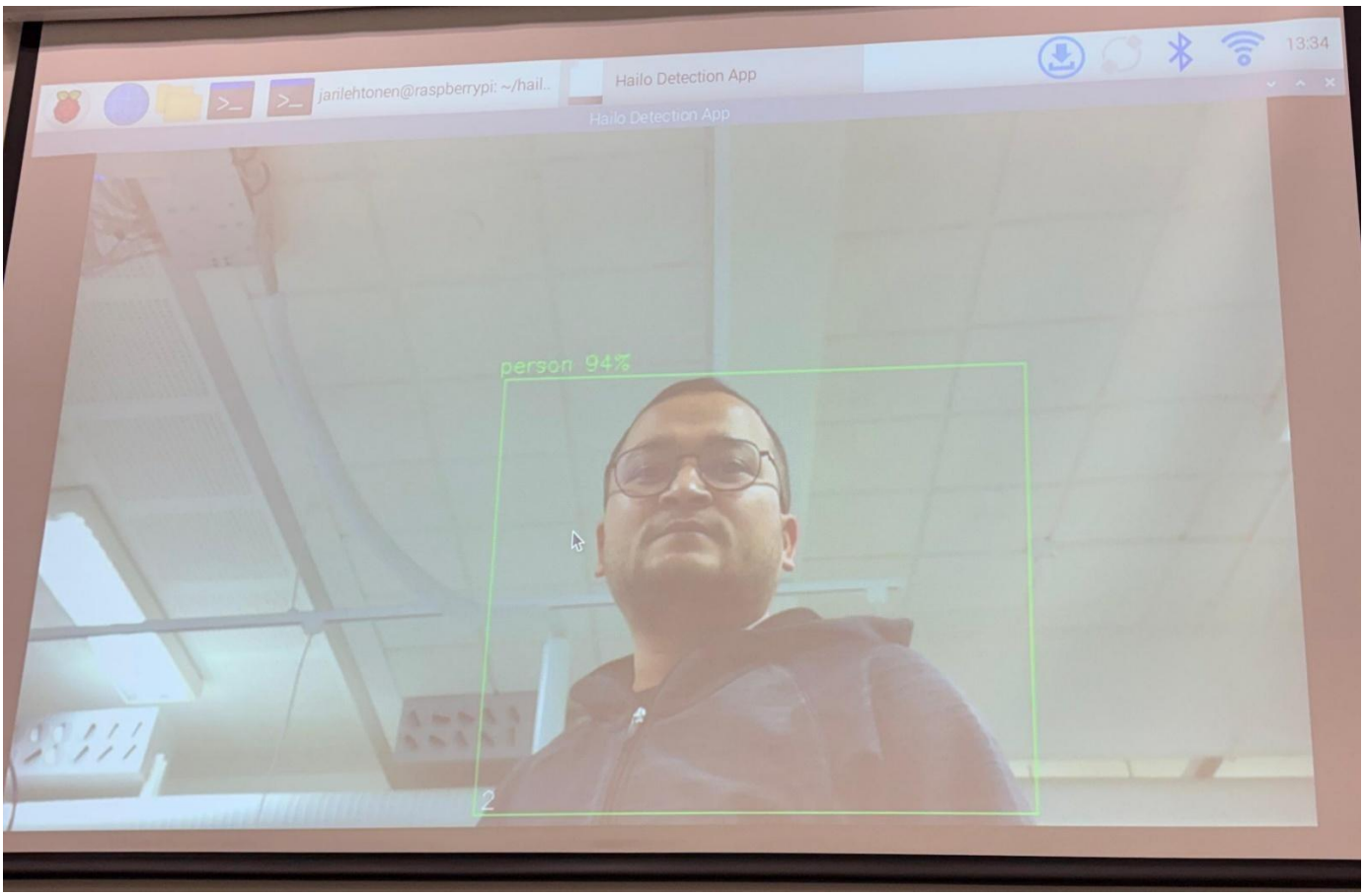
Output in Terminal Task 4:

```
labuser@RiiA103RPiZe21:~/pigpio/milan_udit $ python3 lab7_task4_1.py
Listening for detections...
Connected to MQTT

MQTT Message: {"label": "person", "confidence": 0.94814193248748 78}
Label: person
Confidence: 0.9481419324874878
RED → High confidence person detected

MQTT Message: {"label": "person", "confidence": 0.9481419324874878}
Label: person
Confidence: 0.9481419324874878
RED → High confidence person detected

MQTT Message: {"label": "person", "confidence": 0.388767272233963}
Label: person
Confidence: 0.388767272233963
GREEN → No person detected

MQTT Message: {"label": "person", "confidence": 0.5159341096878052}
Label: person
Confidence: 0.5159341096878052
YELLOW → Low confidence person detected

MQTT Message: {"label": "person", "confidence": 0.82248455286026}
Label: person
Confidence: 0.82248455286026
YELLOW → Low confidence person detected

MQTT Message: {"label": "person", "confidence": 0.772983193397522}
Label: person
Confidence: 0.772983193397522
YELLOW → Low confidence person detected
```

Output in real circuit:



We also used the values from the camera too.



Task 5:

Managing the Quality of the IoT AI Application

The quality of an IoT AI people-detection application is an important factor, especially when the system is used for real-time monitoring and decision-making. Quality in this context does not depend on a single metric, but on several technical and functional criteria working together.

Quality Criteria for the People Detection Application

The most important quality criteria for the people detection application are accuracy, reliability, latency, robustness, and energy efficiency. Accuracy refers to how correctly the AI model detects people and avoids false positives or false negatives. Reliability describes how consistently the system works over time without crashes or communication failures. Latency is the time delay between a person appearing in front of the camera and the detection result being received by the subscriber device. Robustness measures how well the system performs under different lighting conditions, distances, or partial occlusions. Energy efficiency is also important, especially for edge devices, to ensure long-term operation without excessive power consumption.

Measuring the Quality Criteria

The quality criteria of the people detection application can be measured in several ways. Accuracy can be evaluated by comparing AI detections with real-world observations, for example by counting how often a person is correctly detected when present. False detections can also be logged and analysed. Latency can be measured by observing the time difference between a physical event (person entering the camera view) and the LED response on the Raspberry Pi Zero. Reliability can be evaluated by running the system continuously for an extended period and checking for missed messages or MQTT disconnections. Robustness can be tested by changing lighting conditions or camera angles and observing how detection confidence changes.

Improving the Quality if Performance Degrades

If the quality of the people detection application becomes unacceptable, several improvements can be applied. The AI model can be replaced or retrained using a more suitable dataset to improve accuracy. Camera positioning and lighting conditions can be optimized to reduce detection errors. Adjusting confidence thresholds can help reduce false positives or false negatives. Network stability can be improved by ensuring a reliable MQTT connection or adding reconnection logic

in the software. In addition, software updates and system monitoring can help maintain long-term reliability and performance.

Overall, managing quality in an IoT AI application requires continuous monitoring, testing, and optimization of both the AI model and the communication system.

5. Results and Analysis:

The MQTT subscriber successfully received messages in real time when a person appeared in front of the camera connected to the Raspberry Pi 5. JSON parsing worked correctly, and the extracted label and confidence values matched the expected detection behavior.

LED indicators responded accurately:

- When no person was detected, the green LED was ON.
- When a person was detected with high confidence, the red LED turned ON.
- Lower confidence detections activated the yellow LED.
- When the MQTT connection was interrupted, the blue LED indicated the error state.

Minor delays were observed occasionally, mainly due to network latency, but overall system performance was stable and responsive. No major errors occurred during testing.

6. Conclusions:

This lab demonstrated how AI inference results can be integrated into an IoT system using MQTT communication. The experiment successfully combined edge AI, messaging protocols, and physical outputs using LEDs. The lab improved understanding of publish/subscribe architectures, JSON data handling, and real-time system reactions.

Key lessons we learned:

- MQTT is highly suitable for AI-based IoT systems.
- Parsing structured data is essential for reliable system logic.
- Visual indicators help in debugging and monitoring system status.

As future work, the system could be extended by adding logging, multiple detection classes, or a web-based dashboard for visualization.

7. References:

- Microcontrollers & Electronics – Lab 7 Instructions
- HAMK – A Good Lab Report Guidelines

The End